Software Requirement Specification

for a simple billing application

Luca Rosenberg, Andrea Usuelli, Willi Zeltner

25.05.2026

Contents

Revision History

Name	Date	Reason for Change	Version
Luca Rosenberg, Andrea Usuelli, Willi Zeltner	25.05.2026	Initial version	1.0

1. Introduction

This application is a streamlined work reporting system designed for simplicity and ease of use. Its primary purpose is to allow workers (end-users) to document completed tasks associated with specific customers. Accountants (administrative users) can then aggregate these reports to generate accurate invoices.

While many market alternatives offer complex workflows, this application intentionally prioritises a minimal feature set to reduce cognitive overhead and ensure high adoption rates among field staff. This document also outlines out-of-scope concepts and future considerations to provide a comprehensive roadmap without compromising the initial goal of simplicity.

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for a streamlined work reporting and automated invoicing application. Its primary objectives are to provide a clear technical roadmap for developers and testers, to serve as a project baseline for the student group, and to act as a formal evaluation document for the course supervisor. Some chapters therefore extend beyond the standard IEEE SRS guidelines to meet course-specific requirements.

1.2 Product Scope

The application, hereafter referred to as "the system," facilitates the logging of labour and material usage into digital work reports and automates the aggregation of those reports into professional invoices, thereby reducing manual billing errors.

Exclusions: In its current version, the system intentionally excludes authentication and login modules, advanced administrative configuration, integrated mailing services, and a dedicated portal for external customers. These are considered out of scope in order to deliver a lightweight, focused MVP (Minimum Viable Product).

Priority: Requirements listed in Sections 3, 4, and 5 are divided into high and low priority. High-priority requirements must be included in the MVP; low-priority requirements *may* be included if time permits.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface – a set of definitions and protocols for building and integrating application software.
AI/ML	Artificial Intelligence / Machine Learning – techniques that enable systems to learn from data or perform tasks that typically require human reasoning.
Invoice	A billing document generated by aggregating one or more work reports for a specific project or customer.
MVP	Minimum Viable Product – a version of a product with just enough features to be usable by early adopters.
ORM	Object-Relational Mapper – a library that translates between object-oriented code and relational database tables.
SRS	Software Requirements Specification – this document; describes the intended purpose and nature of the software.
Stem Data	A centralised catalogue of available personnel, hourly rates, and material items available for selection in reports.

Term	Definition
UI	User Interface – the visual elements through which a user interacts with the software.
Work Report	A digital record documenting specific completed tasks, including time spent and materials used.

1.4 Project Organisation

This project is carried out by three students in the Master in Life Sciences programme at ZHAW in Wädenswil. It forms part of the final grade in the course "Software Engineering and Design Patterns" and is supervised by Dr. Ahmad Aghaebrahimian.

Team Member	Scrum Role	Project Role
Luca Rosenberg	Development, Software Architecture	Define software architecture and lead its implementation.
Andrea Usuelli	Development, Testing	Full-stack implementation and design of unit tests.
Willi Zeltner	Product Owner	Write software documentation and requirements; verify functional requirements through testing.

1.5 References

The following documents and standards provide additional context for this SRS:

- ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering. Informative.
- Percival, H. J. W. and Gregory, B. *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices.* 2nd ed., O'Reilly, 2020.

1.6 Document Overview

This SRS is organised into six sections. Following this Introduction, **Section 2** provides a high-level view of product functions and user characteristics. **Section 3** details the core functional requirements, grouped by feature area. **Section 4** defines external interface requirements covering UI, hardware, and software integrations. **Section 5** addresses further non-functional requirements including architecture, database, and quality of service. **Section 6** defines the verification approach and environment. All figures and models are provided in the **Appendices** at the end of the document.

2. Product Overview

2.1 Product Perspective

The market for work reporting tools is saturated with feature-rich, complex applications that often overwhelm small teams with unnecessary functionality and come at considerable cost. Many also rely on proprietary cloud storage, making direct data access difficult.

This application is a lightweight alternative specifically tailored for small companies with one to five employees. Although the initial release is intentionally simple, the system's architecture must remain modular to allow for future standalone extensions or integration into larger ecosystems.

2.2 Product Functions

The system's core functionality covers the complete workflow from labour documentation to billing:

- **Work Documentation**: Workers can create, view, and edit daily reports linked to specific customers.
- **Report Management**: Accountants can review and modify all worker reports to ensure data integrity before billing.
- **Automated Invoicing**: Accountants can aggregate one or more reports into a single, professional invoice.
- **Stem Data Utilisation**: All users can draw from a predefined catalogue of materials and human resources to ensure consistency.

2.3 Product Constraints

The following constraints are mandatory for project success and compliance:

- **Open Source**: The application must be developed as open-source software and hosted on a public repository (e.g., GitHub).
- **Minimalist UI**: The interface must allow a worker to complete a standard work report entry in a minimal number of interactions. The MVP UI intentionally forgoes visual polish in favour of delivering functional requirements.
- **Scope**: Not all features required for a production-ready system can be realised within the project timeline. Out-of-scope items are documented in Section 1.2.
- **Standalone**: While a production release would require deployment within a connected web framework, the MVP is designed to run and be evaluated in a local environment (localhost).
- **Timeline**: Development is strictly limited to the duration of the Software Engineering and Design Patterns course in semester SS 2026.

2.4 User Characteristics

Worker (*End-User*)

Attribute	**Description**
Profile	Moderate IT skills. Primary goal is fast data entry under field conditions.
Device	Primarily mobile devices (smartphones).
Access	Can create and edit their own reports. Edit access is revoked once a report has been included in a billed invoice.
Needs	High performance and offline-first capabilities to prevent frustration on site.
Permissions	Can view customers and the stem catalogue but cannot edit them. May add free-text positions for unique items not found in the stem. Unit prices for human resources are hidden to preserve confidentiality.

Accountant (*Administrative User*)

Attribute	**Description**
Profile	Intermediate IT skills. Focuses on data accuracy and financial output.
Device	Primarily desktop systems with large displays.
Access	Full visibility of all worker reports, grouped by customer.
Needs	Bulk editing capabilities and clear status indicators (e.g., Billed vs. Unbilled).

Attribute	Description
Permissions	Can override any report data prior to invoicing and trigger the automated invoice generation process.

2.5 Assumptions and Dependencies

- **Usage**: The system is intended to be used by Willi with no broader deployment. This allows several simplifications:
  - **Database-Level Administration**: Stem data (personnel rates, material costs) will be managed directly at the database level. No UI for these tasks is required at this stage.
  - **Single User**: Strict user permission enforcement is not required at this stage. The scope is nevertheless broadened to include two distinct views (see Section 2.4) in anticipation of future multi-user use.
- **Projects**: Other tools for similar tasks allow work reports to be assigned to projects. It is assumed that the target user's engagements are small enough to assign reports directly to customers, making a separate project entity unnecessary.

3. Main Features

The functional requirements are divided into two feature areas corresponding to the use cases for *workers* and *accountants* (see *Appendix A, Figure 1* and Section 2.4).

Both features rely on the same domain model, whose main entities are:

- **Employees** – workers for whom hours can be reported
- **Customers** – recipients of work reports and invoices
- **Work Reports**
- **Work Report Positions**
- **Invoices**

All of these entities must be present in the MVP. A detailed class diagram with relationships is provided in *Appendix A (Figure 3)*.

3.1 Reporting Work

The core of this feature is the creation of **work reports** (`REQ-FUNC-001`). A report is created by an **employee** and addressed to a **customer**; **work report positions** can then be added to it (`REQ-FUNC-003`). Both the report header and its positions remain editable (`REQ-FUNC-002`, `REQ-FUNC-004`) as long as the report has not been included in a billed **invoice**. Reports that are no longer needed can be soft-deleted (`REQ-FUNC-005`).

High Priority

`REQ-FUNC-001` *(New Report)* A worker can create a new report and associate it with a customer and an employee.

`REQ-FUNC-002` *(Edit Report)* A worker can open an editable report and update the associated customer and employee.

`REQ-FUNC-003` *(Add Report Position)* A worker can open an editable report and add a work report position.

`REQ-FUNC-004` *(Edit Report Position)* A worker can open an editable report and update existing work report positions.

`REQ-FUNC-005` *(Delete Report)* A worker can soft-delete an editable report.

Low Priority

`REQ-FUNC-006` *(Stem Integration)* A worker can select between material, personnel, and free-text position types when adding work report positions. Material and personnel positions are drawn from the stem data catalogue.

`REQ-FUNC-007` *(Work Hours Summary)* A worker can view a summary of reported hours aggregated by day, week, and month.

3.2 Creating Invoices

The core of this feature is the creation of **invoices** (`REQ-FUNC-011`). An invoice is addressed to a **customer** and consolidates the work report positions from one or more **work reports** belonging to that same customer (`REQ-FUNC-013`). The associated work reports and customer can be updated after initial creation (`REQ-FUNC-012`, `REQ-FUNC-014`). Once the invoice is complete, it can be downloaded as a PDF (`REQ-FUNC-015`) and its status set to "Billed" or "Paid" (`REQ-FUNC-016`), at which point the constituent work reports become locked (`REQ-FUNC-017`). Invoices that are no longer needed can be soft-deleted (`REQ-FUNC-018`).

High Priority

`REQ-FUNC-010` *(Basic Accountant Requirements)* All requirements `REQ-FUNC-001` through `REQ-FUNC-006` are also available to accountants.

`REQ-FUNC-011` *(Add Invoice)* An accountant can create a new invoice and associate it with a customer.

`REQ-FUNC-012` *(Edit Invoice)* An accountant can open an invoice and update the associated customer.

`REQ-FUNC-013` *(Add Report to Invoice)* An accountant can open an invoice and add work reports associated with the same customer, provided those reports have not already been included in another invoice.

`REQ-FUNC-014` *(Edit Reports on Invoice)* An accountant can open an invoice and update the associated work reports.

`REQ-FUNC-015` *(Invoice PDF Export)* An accountant can open an invoice and download a PDF version of it.

`REQ-FUNC-016` *(Invoice Status)* An accountant can open an invoice and set its status to "Billed" or "Paid".

`REQ-FUNC-017` *(Status Lock)* Work reports associated with an invoice whose status is "Billed" or "Paid" are no longer editable.

`REQ-FUNC-018` *(Delete Invoice)* An accountant can soft-delete an invoice.

4. External Interfaces

4.1 User Interfaces

As described in Sections 2.3 and 2.4, the user interface must be kept simple and optimised for rapid completion of primary tasks. The Worker UI targets smartphone viewports (`REQ-UI-001`), while the Accountant UI targets desktop viewports (`REQ-UI-002`). Both roles require that core workflows are reachable within three interactions from the dashboard (`REQ-UI-003`, `REQ-UI-005`). Mockups are provided in *Appendix C*.

`REQ-UI-001` *(Mobile Worker Design)* The system shall employ responsive web design optimised for viewport widths of 375 px to 430 px (standard smartphones) for the Worker role.

`REQ-UI-002` *(Desktop Accountant Design)* The system shall employ responsive web design optimised for a viewport width of 1440 px (standard notebook) for the Accountant role.

`REQ-UI-003` *(Work Report Access)* A work report must be creatable within three taps from the dashboard. Work report positions must be addable within three taps from the report overview.

`REQ-UI-004` *(Report Dashboard)* The dashboard shall display the most recent reports requiring attention and the status of the latest sent invoices.

`REQ-UI-005` *(Invoice Access)* An invoice must be creatable within three clicks from the dashboard. Work reports to be billed must be addable within three clicks from the invoice overview.

`REQ-UI-006` *(Export Action)* A downloadable PDF version of an invoice shall be generatable with a single click, based on a predefined template (see `REQ-FUNC-015`).

4.2 Hardware Interfaces

There are no hardware interface requirements for this release.

4.3 Software Interfaces

`REQ-INT-001` *(PDF Generation)* The system shall integrate with a PDF generation library to transform aggregated report data into a standardised invoice format.

`REQ-INT-002` *(Database ORM)* The system shall interface with a relational database via an ORM layer to ensure data persistence and support direct admin-level queries.

5. Further Non-Functional Requirements

5.1 Design and Implementation

5.1.1 Architecture

The system follows a client–server architecture (`REQ-IMP-001`) with a strict separation between the frontend and the backend. The backend is structured according to Domain-Driven Design (DDD) (`REQ-IMP-002`),

organising the codebase around the core business concepts — work reports, invoices, customers, and employees — rather than technical layers alone. Data access and business logic are further decoupled through a service and repository pattern (`REQ-IMP-003`).

These choices are deliberate, even if they may exceed the immediate demands of an MVP. A client–server split enables the frontend and backend to evolve independently, supporting future replacement or extension of either layer without affecting the other. DDD ensures that the domain model remains the authoritative source of truth for business rules, making the codebase easier to reason about as the system grows. The service and repository pattern makes individual components testable in isolation and allows the underlying database to be swapped without touching business logic — relevant given the planned migration from SQLite to PostgreSQL (see Section 5.1.2). Collectively, these patterns reflect the architectural principles studied in the course (Percival & Gregory, 2020).

> `REQ-IMP-001` *(Client–Server Model)* The system shall follow a client–server architecture with a clear separation between the frontend and backend.

> `REQ-IMP-002` *(Domain-Driven Design)* The system shall be implemented using Domain-Driven Design (DDD).

> `REQ-IMP-003` *(Services and Repositories)* The system shall use a service and repository pattern to manage data access and business logic.

5.1.2 Database

The proof-of-concept uses SQLite as its database; PostgreSQL is planned for production. This choice reflects several converging considerations. The system's data (see Section 3 and *Appendix B*) is inherently structured and relational, with well-defined relationships and no unstructured or variable-schema content. A relational database is therefore the natural fit. Both SQLite and PostgreSQL are open-source and free of licensing cost, which aligns with the open-source constraint in Section 2.3. SQLite requires zero configuration and is ideal during development and for the PoC, while PostgreSQL provides the ACID compliance, concurrent write support, and production-grade reliability needed for a deployed system. The transition between the two is made straightforward by the ORM layer (`REQ-INT-002`), which abstracts the database dialect and allows the same model and query code to run against either backend. PostgreSQL is also natively supported by all major free-tier hosting providers, making it compatible with the standalone and cost constraints of this project.

5.1.3 Distribution

The source code is hosted on GitHub in a private repository. Public access is not planned at this time.

Since the product owner is part of the development team and production usage is uncertain at this stage, continuous integration and deployment (CI/CD) is not planned for the current release.

> `REQ-IMP-004` *(Distribution)* The source code shall be accessible on GitHub. The latest commit on the main branch shall always represent a working version of the application.

5.1.4 Proof of Concept and Deadline

> `REQ-DEAD-001` *(Mid-Semester PoC)* A functional proof of concept demonstrating the end-to-end flow from work report creation to invoice generation must be ready for the mid-semester review (date: 25.05.2026).

5.1.5 Change Management

Change management requirements have not yet been specified. Future iterations should define change categories (breaking, additive, bugfix), approval workflows, and required artefacts such as changelogs, migration guides, and release notes. Backward and forward compatibility guarantees, deprecation timelines, and rollout and rollback procedures should also be addressed.

5.2 Quality of Service

5.2.1 Performance

Performance requirements are assigned lower priority for the MVP, as functional correctness takes precedence. Nevertheless, two baseline targets are defined to ensure the system remains usable in the field.

> `REQ-PERF-001` *(Stem List Load Time)* The system shall load the stem selection list (up to 500 items) in under 500 ms under standard 4G network conditions.

> `REQ-PERF-002` *(PDF Generation Time)* PDF invoice generation shall complete within 3 seconds of the accountant's request.

5.2.2 Security

As noted in Section 1.2, authentication is out of scope at this stage. The following requirements address fundamental data integrity and confidentiality concerns that must be present even in the MVP.

> `REQ-SEC-001` *(SQL Injection Prevention)* The system shall use parameterised queries for all database interactions to prevent SQL injection.

> `REQ-SEC-002` *(Soft Deletes and Versioning)* All delete actions shall be implemented as soft deletes. Updates to stem items shall produce a new version of the item, preserving consistency in existing reports.

An implementation of user authentication at a later stage would also enable role-based access control:

> `REQ-SEC-003` *(Role Privacy)* The system shall filter stem data in the Worker view to exclude unit costs and profit margins.

5.3 AI/ML

The system does not incorporate AI or ML components, and this is a deliberate design decision rather than an oversight. The core workflows are fully deterministic and rule-based. Every billing calculation can be expressed as a precise arithmetic formula, and every workflow step follows a defined sequence of user actions. There is no unstructured input, no pattern that needs to be learned from historical data, and no ambiguity that a model would need to resolve. Introducing AI/ML in this context would add infrastructure complexity and maintenance overhead without providing any functional benefit that cannot be achieved more reliably with simple conditional logic.

Beyond the technical argument, the target users are field workers and accountants in small companies who prioritise predictability and transparency. A billing system that produces deterministic, auditable results is more appropriate in this context than one that relies on probabilistic inference. Any unexpected model output would undermine trust in a financial tool.

One area where AI/ML could plausibly add value in a future release is automated text recognition of physical receipts or delivery notes, allowing workers to scan a document and pre-populate a work report position rather than entering data manually. This capability is noted in the project roadmap but is explicitly out of scope for the current version.

6. Verification

This section outlines the objective evidence required to confirm that each requirement has been satisfied. The primary verification methods are **Test** (automated or manual execution), **Analysis** (evaluation of code or logic), and **Demonstration** (walkthrough with a stakeholder).

Requirement ID	Label	Implementation Status	Verification Method	Verification Status
REQ-FUNC-001	New Report	Prototype	Demonstration	Passed
REQ-FUNC-002	Edit Report	Prototype	Demonstration	Passed
REQ-FUNC-003	Add Report Position	Prototype	Demonstration	Passed
REQ-FUNC-004	Edit Report Position	Prototype	Demonstration	Passed
REQ-FUNC-005	Delete Report	Prototype	Demonstration	Passed
REQ-FUNC-006	Stem Integration	Prototype	Demonstration	Passed
REQ-FUNC-007	Work Hours Summary	Prototype	Demonstration	Passed
REQ-FUNC-010	Basic Accountant Requirements	Prototype	Demonstration	Passed
REQ-FUNC-011	Add Invoice	Prototype	Demonstration	Passed
REQ-FUNC-012	Edit Invoice	Prototype	Demonstration	Passed
REQ-FUNC-013	Add Report to Invoice	Prototype	Demonstration	Passed
REQ-FUNC-014	Edit Reports on Invoice	Prototype	Demonstration	Passed
REQ-FUNC-015	Invoice PDF Export	Prototype	Demonstration	Passed
REQ-FUNC-016	Invoice Status	Prototype	Demonstration	Passed
REQ-FUNC-017	Status Lock	Prototype	Demonstration	Passed
REQ-FUNC-018	Delete Invoice	Prototype	Demonstration	Passed
REQ-PERF-001	Stem List Load Time	Prototype	Test (Manual)	Not yet tested
REQ-PERF-002	PDF Generation Time	Prototype	Test (Manual)	Not yet tested
REQ-SEC-001	SQL Injection Prevention	Work In Progress	Analysis	—
REQ-SEC-002	Soft Deletes and Versioning	Prototype	Analysis	Passed
REQ-SEC-003	Role Privacy	Pending	Demonstration	—
REQ-UI-001	Mobile Worker Design	Work In Progress	Demonstration	—
REQ-UI-002	Desktop Accountant Design	Prototype	Demonstration	Passed
REQ-UI-003	Work Report Access	Prototype	Demonstration	Partially passed
REQ-UI-004	Report Dashboard	Pending	Demonstration	—
REQ-UI-005	Invoice Access	Prototype	Demonstration	Partially passed
REQ-UI-006	Export Action	Prototype	Demonstration	Passed
REQ-INT-001	PDF Generation	Prototype	Analysis	Passed
REQ-INT-002	Database ORM	Prototype	Analysis	Passed
REQ-IMP-001	Client–Server Model	Prototype	Analysis	Passed

Requirement ID	Label	Implementation Status	Verification Method	Verification Status
REQ-IMP-002	Domain-Driven Design	Prototype	Analysis	Passed
REQ-IMP-003	Services and Repositories	Prototype	Analysis	Passed
REQ-IMP-004	Distribution	Prototype	Demonstration	Partially passed
REQ-DEAD-001	Mid-Semester PoC	Prototype	Demonstration	Passed

6.1 Verification Environment

- **Development:** Localhost (Python environment).
- **Staging:** Private repository, local testing.
- **Database:** SQLite for local testing.
- **Tools:** PyTest for logic testing.

6.2 Unit Tests

Automated unit tests are implemented for two areas of the codebase. The first and larger set covers input validation for data submitted from the frontend. Those inputs are asserted to conform to expected formats and constraints before being processed by the backend. Invalid inputs are expected to be rejected with appropriate error responses, and the tests verify this behaviour across a range of valid, boundary, and malformed values. The second, smaller set of tests covers the report locking logic defined in REQ-FUNC-017. These tests confirm that a work report associated with an invoice in "Billed" or "Paid" status cannot be modified, and that the lock is applied correctly regardless of the update pathway used.

6.3 Maintenance

With the conclusion of the course "Software Engineering and Design Patterns," Luca Rosenberg and Andrea Usuelli leave the project team. Product owner Willi Zeltner would be the sole remaining contributor for any future development. The departure of the two primary developers introduces several concrete risks. Domain knowledge about the architecture, data model, and implementation decisions is largely held by Luca and Andrea. Eventough it is captured in this SRS and the repository documentation without them, diagnosing bugs or extending the system would require a significant onboarding effort for any new contributor. The risks can be mitigated to a reasonable degree by a focused handover and this documentation. Concretely, this means the architecture documentation in *Appendix B* gives valuable insights, UML models in *Appendix A* help understand decisions and implementation. As a next step, any known limitations or deferred design decisions should be recorded as issues in the GitHub repository so that the state of the project is transparent to anyone picking it up later. Given that production deployment is not currently planned (see Section 2.5), the immediate maintenance burden is low. Still, these steps would make the codebase significantly more approachable should the system be extended or deployed in the future.

Appendix A: Analysis Models

The models below can also be found in the GitHub repository under `documentation/diagrams`.

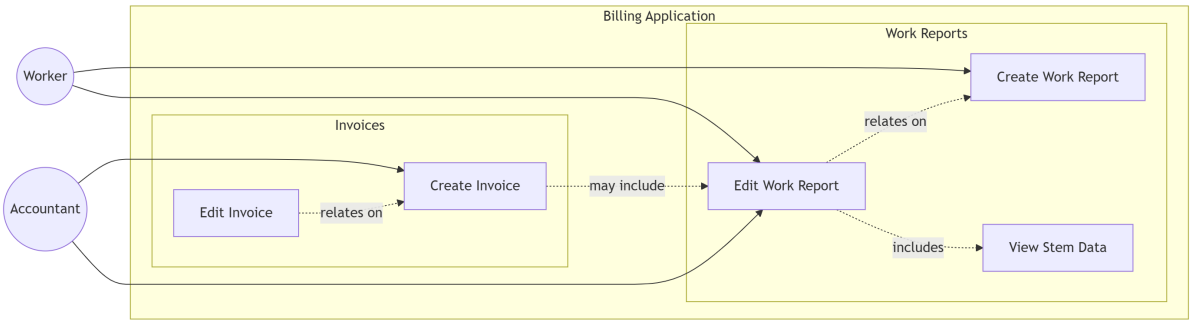

Figure 1: Use Case diagram. The relevant actors are the worker and the accountant.

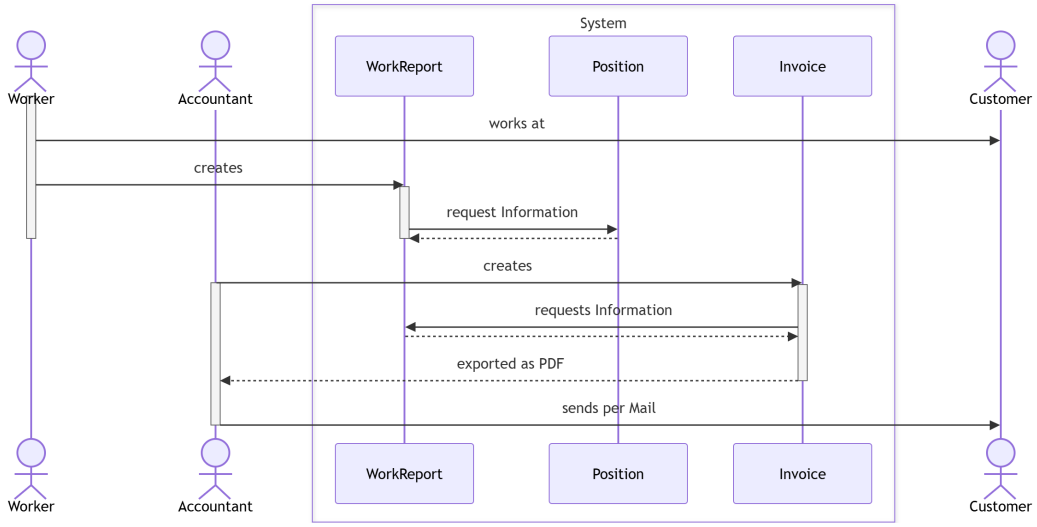

Figure 2: Sequence diagram. Divided between worker and accountant workflows.

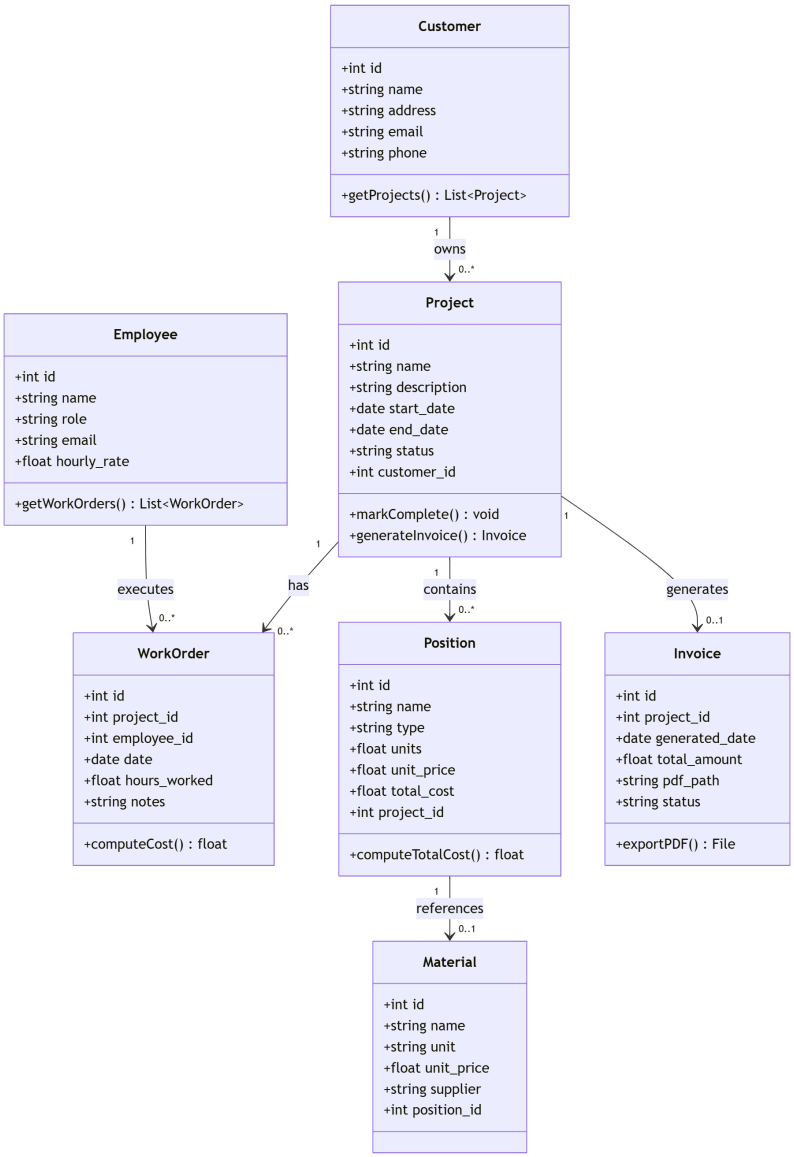

Figure 3: Base classes identified. This model excludes service and repository classes as well as API routes.

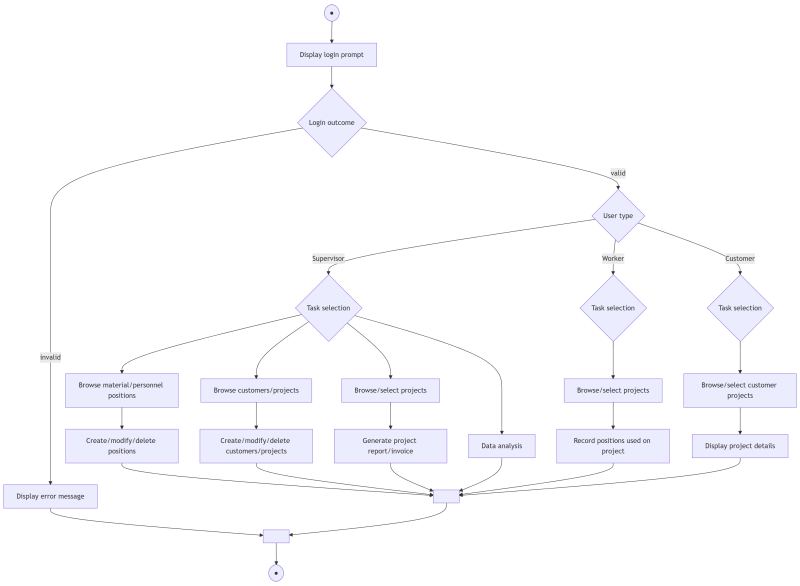

Figure 4: Activity diagram.

Appendix B: Architecture Models

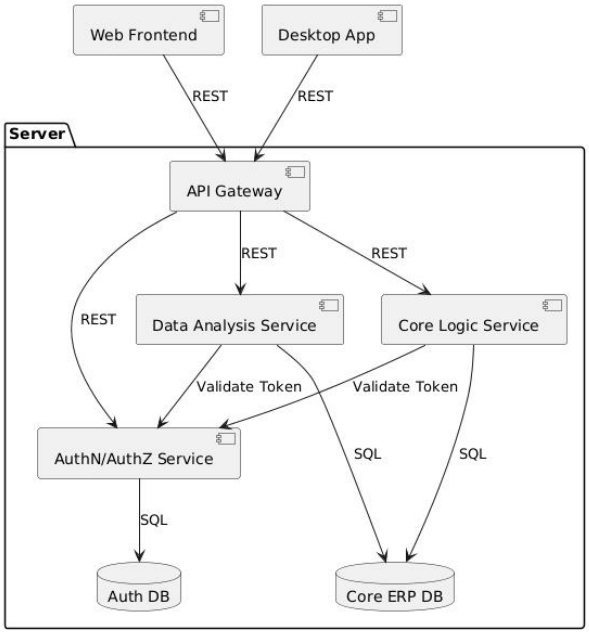

Figure 5: Architecture design.

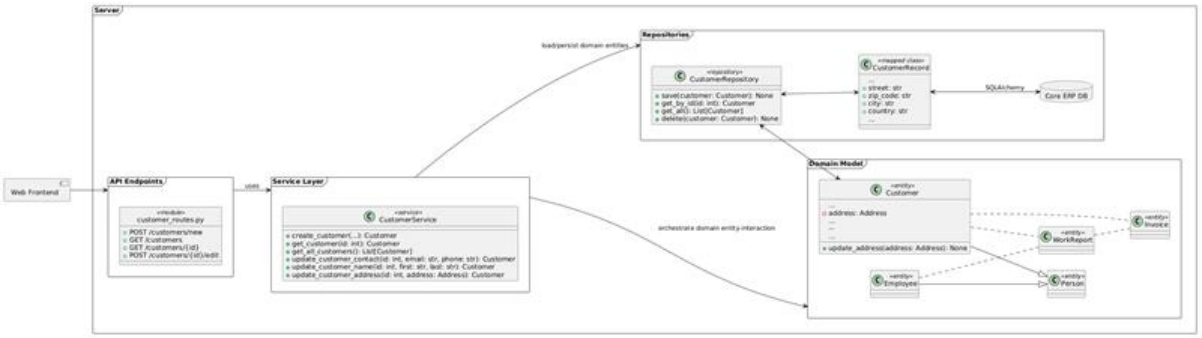

Figure 6: Simplified illustration of the Domain-Driven Design structure. This pattern applies to every domain class shown in *Figure 3 (Appendix A)*.

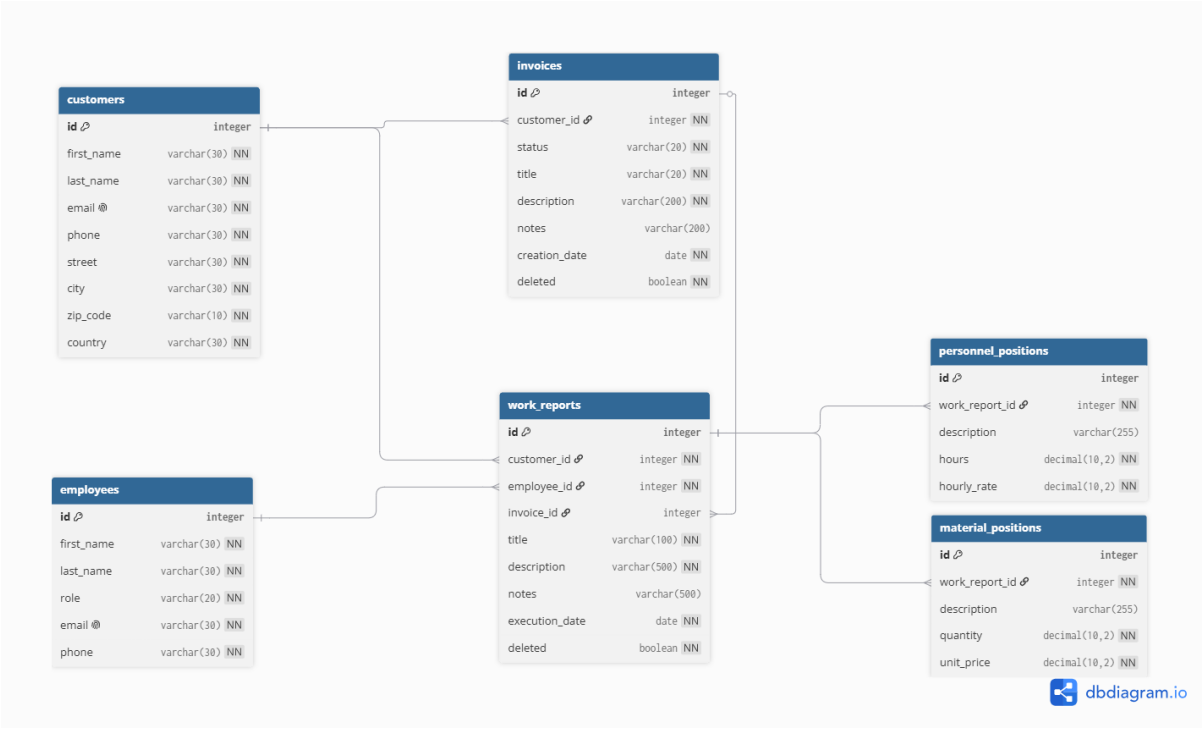

Figure 7: Entity–Relation diagram

Appendix C: User Interface Mockups

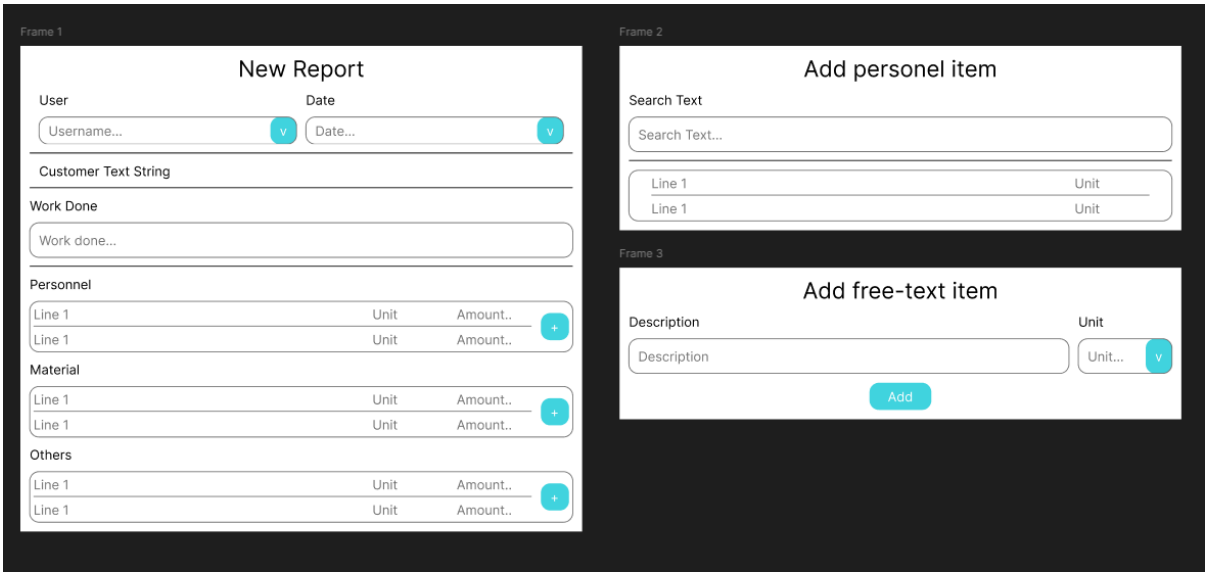

Figure 8: Mockup for the worker UI when adding a new report. Smaller frames represent individual position entries to be added to a work report.

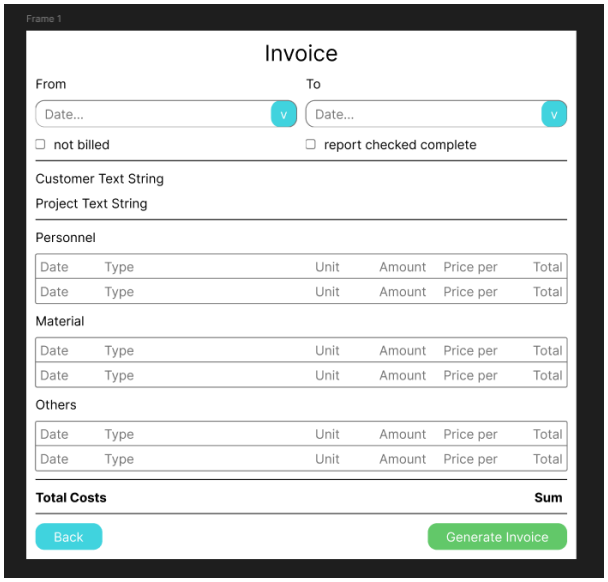

Figure 9: Mockup for the accountant UI when generating a new invoice. Shows an overview of relevant positions and reports, the invoice total, and the action buttons.